

Dataflow templates

Dataflow templates allow you to package a Dataflow pipeline for deployment. Anyone with the correct permissions can then use the template to deploy the packaged pipeline. You can create your own custom Dataflow templates, and Google provides [pre-built templates](#) ([dataflow/docs/templates/provided-templates](#)) for common scenarios.

Benefits

Templates have several advantages over directly deploying a pipeline to Dataflow:

- Templates separate pipeline design from deployment. For example, a developer can create a template, and a data scientist can deploy the template at a later time.
- Templates can have parameters that let you customize the pipeline when you deploy the template.
- You can deploy a template by using the Google Cloud console, the Google Cloud CLI, or REST API calls. You don't need a development environment or any pipeline dependencies installed on your local machine.
- A template is a code artifact that can be stored in a source control repository and used in continuous integration (CI/CD) pipelines.

Google-provided templates

Google provides a variety of pre-built, open source Dataflow templates that you can use for common scenarios. For more information about the available templates, see [Google-provided templates](#) ([dataflow/docs/templates/provided-templates](#)).

Starting August 4, 2025, Google-managed template jobs run on [Dataplex Edge](#) ([dataflow/docs/guides/enable-dataflow-prime](#)) by default. To run your job on standard Dataflow, you must disable Dataflow Prime. To disable Dataflow Prime for Google-managed template jobs, set the `enable_prime` Dataflow service option to `false` (`enable_prime=false`) when you run the template. For more details, see [Disable Dataflow Prime](#) ([dataflow/docs/guides/enable-dataflow-prime#disable-prime](#)).

Compare Flex templates and classic templates

Dataflow supports two types of template: Flex templates, which are newer, and classic templates. If you are creating a new Dataflow template, we recommend creating it as a Flex template.

With a Flex template, the pipeline is packaged as a Docker image in Artifact Registry, along with a template specification file in Cloud Storage. The template specification contains a pointer to the Docker image. When you run the template, the Dataflow service starts a launcher VM that pulls the Docker image, and runs the pipeline. The execution graph is dynamically built based on runtime parameters provided by the user. To use the API to launch a job that uses a Flex template, use the [executeTemplate](#) ([dataflow/docs/reference/rest/v1/projects/locations/templates/executeTemplate](#)) method.

A classic template contains the JSON serialization of a Dataflow job graph. The code for the pipeline must wrap any runtime parameters in the `ValueProvider` interface. This interface allows users to specify parameter values when they deploy the template. To use the API to work with classic templates, see the [executeTemplate](#) ([dataflow/docs/reference/rest/v1/projects/locations/templates/executeTemplate](#)) API reference documentation.

Flex templates have the following advantages over classic templates:

- Unlike classic templates, Flex templates don't require the `ValueProvider` interface for input parameters. Not all Dataflow sources and sinks support `ValueProvider`.
- While classic templates have a static job graph, Flex templates can dynamically construct the job graph. For example, the template might select a different I/O connector based on input parameters.
- A Flex template can perform preprocessing on a virtual machine (VM) during pipeline construction. For example, it might validate input parameter values.

Template workflow

Using Dataflow templates involves the following high-level steps:

1. Developers set up a development environment and develop their pipeline. The environment includes the Apache Beam SDK and other dependencies.
2. Depending on the template type (Flex or classic):
 - For Flex templates, the developers package the pipeline into a Docker image, push the image to Artifact Registry, and upload a template specification file to Cloud Storage.
 - For classic templates, developers run the pipeline, create a template file, and stage the template to Cloud Storage.
3. Other users submit a request to the Dataflow service to run the template.
4. Dataflow creates a pipeline from the template. The pipeline can take as much as five to seven minutes to start running.

Set IAM permissions

Dataflow jobs, including jobs run from templates, use two IAM service accounts:

- The Dataflow service uses a [Dataflow service account](#) ([dataflow/docs/concepts/security-and-permissions#service-account](#)) to manipulate Google Cloud resources, such as creating VMs.
- The Dataflow worker VMs use a [worker service account](#) ([dataflow/docs/concepts/security-and-permissions#worker-service-account](#)) to access your pipeline's files and other resources. This service account needs access to any resources that the pipeline job references, including the source and sink that the template uses. For more information, see [Access Google Cloud resources](#) ([dataflow/docs/concepts/security-and-permissions#access-resources](#)).

Ensure that these two service accounts have appropriate roles. For more information, see [Dataflow security and permissions](#) ([dataflow/docs/concepts/security-and-permissions](#)).

Apache Beam SDK version requirements

To create your own templates, make sure your Apache Beam SDK version supports template creation.

Java/Python (Python)

To create templates with the Apache Beam SDK 2.x for Java, you must have version 2.0.0-beta3 or higher.

To run templates with Google Cloud CLI, you must have [Google Cloud CLI](#) ([sdk/downloads](#)) version 138.0.0 or higher.

Extend templates

You can build your own templates by extending the [open-source](#) ([https://github.com/googlecloudplatform/dataflow-templates](#)) Dataflow templates. For example, for a template that uses a fixed window duration, data that arrives outside of the window might be discarded. To avoid this behavior, use the template code as a base, and modify the code to invoke the [withAllowedStreams](#) ([https://beam.apache.org/documentation/programming-guide/#managing-late-data](#)) operation.

What's next

- [Google-provided templates](#) ([dataflow/docs/templates/provided-templates](#))
- [Use Flex templates to package pipelines](#) ([dataflow/docs/templates/provided-templates](#))
- [Build an example Flex Template](#) ([dataflow/docs/guides/templates/configuring-flex-templates](#))
- [Creating classic templates](#) ([dataflow/docs/templates/creating-templates](#))
- [Running classic templates](#) ([dataflow/docs/templates/running-templates](#))
- [Troubleshoot Flex Templates](#) ([dataflow/docs/guides/troubleshoot-templates](#))

Copyright 2025 Google LLC. The content of this page is licensed under the [Creative Commons Attribution 4.0 License](#) ([https://creativecommons.org/licenses/by/4.0/](#)), and code samples are licensed under the [Apache 2.0 license](#) ([https://www.apache.org/licenses/LICENSE-2.0](#)). For details, see the [Google Developers Site Policies](#) ([https://developers.google.com/site-policies](#)). Java is a registered trademark of Oracle and/or its affiliates. Last updated: 2025-04-02 (UTC).